

Reviewer Pack — Minimal Order of Quaternionic K-theory and Circuit Monotone ...

Entry d250507a8cfc · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-05 00:01:19 UTC

Minimal Order of Quaternionic K-theory and Circuit Monotone Width Inequality

Entry ID: d250507a8cfc **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-05 00:01:19 UTC

1. Conjecture

Field A (mathematical branch): Algebra (Quaternionic K-theory) **Field B** (complexity object): Boolean Circuit Complexity (Circuit Monotone Width)

Statement:

For all instances with a circuit size s , the minimal order of quaternionic K-theory associated with the instance is $O(s^2)$, such that $\min_order(K) = \Theta(s^2)$.

Rationale (proposer's reasoning):

Quaternionic K-theory provides a noncommutative generalization of the classical K-theory, which could potentially offer new insights into the complexity of circuits. The monotone width of a circuit measures its sensitivity to input changes, and an inequality involving both quantities might reveal a deep connection between algebraic structures and computational complexity.

Taxonomy category: QuaternionicKtheory (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): ead035ee3779f8e0

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

For a given CNF formula size s , the minimal order of quaternionic K-theory $\min_order(K)$ is considered supported if it satisfies $\min_order(K) \leq 1.5 * s^2$ for all seeds and is falsified if any seed results in $\min_order(K) > 2 * s^2$.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: ADJACENT_OK against 2 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "Quaternionic K-theory" AND "circuit monotone width" - "minimal order of quaternionic K-theory" AND "circuit size" - " $\Theta(s^2)$ " AND "quaternionic K-theory" AND "Boolean circuit complexity"

Top relevant hits considered: - [s2:10.7916/D8ZP45KT] Unconditional Lower Bounds in Complexity Theory - [s2:10.1007/11537311] Fundamentals of Computation Theory, 15th International Symposium, FCT 2005, Lübeck, Germany, August 17-20, 2005, Proceed

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_cnf_formula(n):
        clauses = []
        for _ in range(n):
            clause = [random.randint(1, n), -random.randint(1, n)]
            clauses.append(clause)
        return clauses

    def construct_quaternionic_matrix(cnf_formula):
        n = len(cnf_formula)
        Q = [[0] * (n + 2) for _ in range(n + 2)]
        for clause in cnf_formula:
            i, j = abs(clause[0]) - 1, abs(clause[1]) - 1
            if clause[0] > 0:
                Q[i][j + n + 1] = 1
            else:
                Q[j][i + n + 1] = 1
            Q[i][n] += 1
            Q[j][n] += 1
        return Q

    def determinant(matrix):
        n = len(matrix)
        if n == 2:
```

```

        return matrix[0][0] * matrix[1][1] - matrix[0][1] * matrix[1][0]
    det = 0
    for j in range(n):
        submatrix = [row[:j] + row[j+1:] for row in matrix[1:]]
        det += (-1) ** j * matrix[0][j] * determinant(submatrix)
    return det

def compute_min_order_K(cnf_formula):
    Q = construct_quaternionic_matrix(cnf_formula)
    det_Q = determinant(Q)
    min_order_K = abs(det_Q)
    return min_order_K

n_max = 40
instances_tested = 30
total_metric_value = 0.0
conjecture_holds = True
counterexample = ""

for _ in range(instances_tested):
    s = random.randint(5, 40)
    cnf_formula = generate_cnf_formula(s)
    min_order_K = compute_min_order_K(cnf_formula)

    if min_order_K > 2 * s**2:
        conjecture_holds = False
        counterexample = f"CNF size {s}, min_order(K) = {min_order_K}"
        break

    total_metric_value += min_order_K

mean_metric_value = total_metric_value / instances_tested
std_metric_value = (sum((x - mean_metric_value)**2 for x in [compute_min_order_K(generate_cnf_formula(s)) for s in range(5, n_max + 1)])) ** 0.5

return {
    "metric_name": "min_order(K)",
    "metric_value": mean_metric_value,
    "instances_tested": instances_tested,
    "n_max": n_max,
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    import sys
    seeds = [int(x) for x in sys.argv[1:]] if len(sys.argv) > 1 else [2**i - 1 for i in range(5, 31)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_metric_value = sum(r["metric_value"] for r in results) / len(results)
    std_metric_value = (sum((r["metric_value"] - mean_metric_value)**2 for r in results)) ** 0.5
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

```

```

if all(r["conjecture_holds"] for r in results):
    print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fraction={support_fraction}")
elif any(not r["conjecture_holds"] for r in results):
    first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
    print(f"RESULT: FALSIFIED counterexample=\"{result['counterexample']}\" first_failing_seed={first_failing_seed}")
else:
    print("RESULT: INCONCLUSIVE reason=unknown")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_f9c9e05e.py", line 93, in <module>
    result = run_trial(seed)
              ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_f9c9e05e.py", line 66, in run_trial
    min_order_K = compute_min_order_K(cnf_formula)
                  ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_f9c9e05e.py", line 52, in compute_min_order_K
    Q = construct_quaternionic_matrix(cnf_formula)
        ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_f9c9e05e.py", line 34, in construct_quaternionic_matrix
    Q[i][j + n + 1] = 1
    ~~~~^~~~~~
IndexError: list assignment index out of range

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data, which means it did not complete the computation to verify the conjecture. | next: Investigate and fix the crash in the test code to ensure it can complete the computation for further verification of the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	12249
2	propose	ollama_remote	glm4:latest	0	0	12187

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
3	preregistration	ollama_remote	glm4:latest	0	0	9347
4	novelty	ollama_remote	glm4:latest	0	0	9005
5	novelty	ollama_remote	glm4:latest	0	0	9752
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	18868
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11562
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11523
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11297
10	judge	ollama_remote	glm4:latest	0	0	12446

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 118236 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in `research/audit/d250507a8cfc.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/d250507a8cfc.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/d250507a8cfc.tar.gz` (if generated)